

Architecture Report: Oman Legal GraphRAG Pipeline

Executive Summary

This project builds an end-to-end GraphRAG pipeline for Omani legal documents from qanoon.om, with English translations from decree.om when they are linked from the Arabic source page. The implementation favors deterministic extraction over unnecessary LLM calls. The crawler uses WordPress REST endpoints when available, stores resumable state in SQLite, serializes content to Markdown, loads consolidated multilingual Document nodes into Neo4j, extracts topics, creates semantic chunks, embeds topics and chunks, and exposes a hybrid search client with both local JSONL and live Neo4j backends.

The implementation is designed to be practical and reproducible: complete enough to run end to end, but intentionally avoids overbuilding speculative infrastructure. The strongest engineering choices are the REST-first crawl gate, exact parsing of site-provided metadata, deterministic legal edge extraction, and a runnable sample dataset.

Crawling And Serialization

The first command checks qanoon.om/wp-json/wp/v2/posts?per_page=1 and decree.om/wp-json/wp/v2/posts?per_page=1. If both return JSON objects with id, the crawler uses WordPress REST pagination. If either endpoint fails, sitemap fallback is available. This avoids expensive browser automation for the normal case while still preserving a recovery route.

The crawler includes practical robustness controls: rotating user agents, randomized pacing, retries with exponential backoff, Retry-After handling, optional proxy configuration, persistent cookie storage, and SQLite replay of pending/failed slugs before new discovery work. A document is marked complete only after its JSONL and Markdown artifacts are written. An optional Playwright detail-page fallback is available when direct HTTP fetches fail, but the primary path remains the public REST API. The implementation does not claim CAPTCHA solving, TLS fingerprint spoofing, or proxy-pool evasion.

Qanoon remains the source of truth. The crawler does not derive decree.om URLs from qanoon slugs because the live site has inconsistent zero padding and different patterns for non-royal-decree documents. Instead, it extracts the embedded English link directly from each Arabic post and follows that exact URL.

Official gazette posts such as og1649 are treated as index pages. They are useful for discovering linked decrees and decisions, but they are not ingested as root Document nodes because they duplicate or aggregate content without being a full legal text.

HTML is converted to Markdown with scripts, style tags, tracking markup, and layout noise removed. PyMuPDF is included as a PDF fallback for documents where HTML is absent or too thin. Arabic PDF extraction is treated as a known risk because right-to-left text, ligatures, and encoding can degrade quality.

Metadata And Graph Modeling

The graph schema keeps each legal instrument consolidated:

- Document stores Arabic and English Markdown as contentAr and contentEn.
- Chunk stores semantic text windows linked by HAS_CHUNK.
- Topic stores extracted legal concepts linked by HAS_TOPIC.
- Legal cross-document edges are REFERENCES, REPEALS, and AMENDS.

Primary document numbers are parsed from the specific paragraph immediately below h1.entry-title, which contains all four formats such as 2026/60 60/2026 ???/?? ??/???. The parser does not scan the body for primary numbers, because legal body text contains many referenced

decree numbers.

Issuer metadata for ministerial decisions is parsed from PDF URLs like `data.qanoon.om/ar/md/{issuer}/2026-0074.pdf`. This is more reliable than asking an LLM to infer the issuer from prose.

Cross-reference edges are extracted from inline `qanoon.om` links and local Arabic legal keywords. `???` `????????` `???` maps to REFERENCES; `????` maps to REPEALS; and `????`, `????`, `???????`, and `????` map to AMENDS. This gives exact, auditable graph edges without topic-model hallucination.

LLM, Embeddings, And Retrieval

The LLM is constrained to topic extraction and optional final answer synthesis. Google Gemini is supported directly through the Generative Language REST API. Topic extraction receives a seed taxonomy and returns JSON. If no API key is configured, a deterministic keyword fallback keeps the pipeline runnable.

The default embedding model is `intfloat/multilingual-e5-small`, chosen because Arabic coverage matters more here than marginal English-only gains. Chunks are formed around Markdown headings and then windowed to roughly 500-900 tokens with overlap.

The search client implements three retrieval stages in both local and Neo4j-backed modes:

1. Candidate generation with dense vector scores and sparse keyword/full-text scores. The default dense weight is 0.35 because the current legal sanity checks showed exact sparse matches are more reliable for title-heavy ministerial decisions than uncalibrated dense scores.
2. Graph expansion by appending parent document metadata, linked topics, and legal references. In Neo4j mode this expansion is performed with live Cypher traversals from retrieved Chunk nodes to their parent Document, Topic, and legal-reference relationships.
3. Optional cross-encoder reranking with `BAAI/bge-reranker-base`.

Final synthesis uses Gemini when `GOOGLE_API_KEY` is configured, with an OpenAI-compatible fallback also available. Without an LLM key, the client returns an extractive answer with source titles and snippets.

Bonus Features And Limits

Topic merging is implemented as a vector-similarity audit with a default threshold of 0.88. The threshold is intentionally conservative: it catches close synonyms while reducing the risk of merging adjacent but legally distinct concepts.

Community detection is included as exploratory analysis via Louvain over the document-topic-reference graph. Each community can be summarized by the configured LLM using its topic labels and representative document titles, and the graph loader can persist `communityId` and `communitySummary` onto Document and Topic nodes. It is not treated as the main quality claim because sparse legal citation graphs can cluster by issuer, time, or publication pattern instead of substantive legal field.

Cross-encoder reranking is optional. It can improve precision, but it is slower than bi-encoder retrieval. The intended production optimization path would be async candidate generation, cached graph expansion, batched reranker inference, and quantized reranker weights.

Retrieval Evaluation

The six-document sample is only a smoke test. To avoid overstating quality, a separate

evaluation harness runs labeled known-answer queries and reports Hit@1, Hit@3, and Mean Reciprocal Rank. A broader local run inspected 80 live Qanoon posts and produced 67 full documents after skipping official-gazette index pages. The set contained 24 royal decrees and 43 ministerial decisions, with 247 chunks, 171 topic links, and 271 extracted cross-reference mentions.

On six positive known-answer queries over this 67-document set, retrieval found the expected document at rank 1 for all six queries. These included health specialties, sports entities, urban planning, postal services, judicial enforcement for Ministry of Commerce officers, and the A'Sharqiya University branch decision. This is a useful sanity check, not a complete benchmark: the queries are hand-labeled, small in number, and partly based on recently crawled documents.

The negative-control query, "Which Omani decree regulates mining colonies on Mars?", still returns nearest neighbors because the current search client always ranks available candidates. A production system needs a calibrated no-answer threshold using held-out negative queries and score distributions.

Another important limitation is legal currentness. The graph stores REPEALS and AMENDS edges, but the retrieval client does not yet demote repealed laws or compute a consolidated current-law status. A production legal search system should mark repealed/amended instruments, prefer current law by default, and still expose historical law when the query asks for it.

Scaling Bottlenecks

The main bottlenecks are crawl time under polite rate limits, Arabic PDF extraction quality, LLM topic extraction cost, embedding throughput, and Neo4j write volume. At larger scale, ingestion should split into independent queues: discovery, fetch, parse, graph upsert, topic extraction, chunking, and embedding. The current loader batches documents, chunks, topics, legal references, and community assignments; at larger scale those batches should be queue-driven with retryable write jobs. Embeddings should run on GPU workers or a managed embedding service.

The system deliberately avoids CAPTCHA bypass. If throttling or CAPTCHA appears, the crawler checkpoints progress, records the failing URL, and pauses for later resume.

Verification

The repository includes offline tests for exact number parsing, English-link extraction, gazette skipping, PDF issuer parsing, Markdown cleanup, chunking, crawl-state resume records, fallback topic detection, and legal edge classification. These tests are useful but intentionally narrow; they do not prove Arabic PDF quality, cross-encoder quality, embedding quality, or end-to-end Neo4j correctness under all failure modes.

Live checks performed during development covered the WordPress REST probe, Gemini smoke completion, Neo4j Aura connectivity, schema/index creation, sample graph loading, vector/full-text index availability, and the 67-document retrieval sanity check. Gemini batch topic extraction can hit provider rate limits; when that happens the topic extractor falls back to deterministic taxonomy matching rather than failing the pipeline.